picoCTF Write-Up

Wireshark doo dooo do doo... — TFTP Forensics Challenge

Category: Forensics | Points: 50 | Lost Boys Cyber

Challenge	Wireshark doo dooo do doo...
Category	Forensics
Points	50
File	tftp.pcapng (~50 MB)
Tools Used	tshark / Wireshark, Python, steghide, dpkg
Techniques	TFTP packet reassembly, ROT13 decoding, Debian package extraction, Steganography
Flag Format	picoCTF{...} (obfuscated below)

Challenge Description

> "Can you find the flag in this packet capture? Download the file and use Wireshark to look inside."

We're given a 50 MB PCAP file: tftp.pcapng. The name of the challenge is a hint — TFTP (Trivial File Transfer Protocol) is a simple, unauthenticated, UDP-based file transfer protocol. The goal is to reconstruct what was transferred and find the hidden flag.

Step 1 — Identify the Protocol and Transfers

Opening the PCAP in Wireshark and filtering for TFTP traffic reveals a series of file transfers between two endpoints. TFTP uses UDP port 69 for initial read/write requests, then negotiates a random ephemeral port for each transfer's data blocks.

```
tshark -r tftp.pcapng -Y "tftp" | head -30
```

Scanning through the Read Requests (RRQ opcodes) reveals four distinct file transfers:

- instructions.txt — a small text file
- plan — another text file
- program.deb — a Debian package archive
- picture1.bmp — a bitmap image

Step 2 — Extract the Transferred Files

TFTP transfers data in 512-byte blocks over UDP. Each transfer session uses a unique destination port, so we can isolate each file by filtering on that port and reassembling the hex-encoded data payloads:

```
tshark -r tftp.pcapng \
  -Y "udp.dstport == <DATA_PORT> && data" \
```

```
  -T fields -e data.data \
  | tr -d '\n' | xxd -r -p > recovered_file
```

Running this for each session recovers all four files intact. Each TFTP data packet carries a 4-byte header (opcode + block number) before the payload, so we strip the first two bytes of each block during reassembly.

Step 3 — Decode instructions.txt

The recovered instructions.txt contains what appears to be random text:

```
Gur synt vf uvqqra va gur cvpgher.
Hfr fgrtuver jvgu cnffcuenfr: QHRQVYVTRAPR
```

This is ROT13 — a Caesar cipher that substitutes each letter with the one 13 positions ahead in the alphabet. It's its own inverse: applying ROT13 twice returns the original text. Decoding with Python:

```
python3 -c "import codecs; print(codecs.decode(open('instructions.txt').read(),
'rot_13'))"
```

> Decoded: "The flag is hidden in the picture. Use steghide with passphrase: DUEDILIGENCE"

We now have three critical pieces of information: the flag is inside picture1.bmp, the tool is steghide, and the passphrase is DUEDILIGENCE.

Step 4 — Decode the plan File

The plan file is also ROT13 encoded. Decoding it gives:

> "I used steghide to hide the flag in picture1.bmp. The passphrase is DUEDILIGENCE."

This confirms the approach and reinforces that steghide was the tool used to embed the data. The attacker (challenge author) stored both the tool and the credentials for recovering it in the same transfer — a classic operational security failure that makes our job easy.

Step 5 — Extract steghide from program.deb

The program.deb file is a Debian package containing the steghide binary and its dependencies. We extract it without requiring root or dpkg installation using the -x flag:

```
dpkg -x program.deb ./steghide_extracted/
# Binary lands at: ./steghide_extracted/usr/bin/steghide
```

steghide depends on libjpeg.so.62 and libmcrypt.so.4, which may not be present on the system. If missing, download the Debian packages for each (.deb files from Debian/Ubuntu mirrors), extract them the same way with dpkg -x, and point the linker to them:

```
export LD_LIBRARY_PATH=./libjpeg_extracted/usr/lib/x86_64-linux-gnu:\
                       ./libmcrypt_extracted/usr/lib:$LD_LIBRARY_PATH
```

Step 6 — Extract the Hidden Data

With steghide available and the passphrase from Step 3, we extract the embedded data from the BMP image:

```
./steghide extract -sf picture1.bmp -p DUEDILIGENCE
# steghide confirms embedded data and extracts a file containing the flag
```

steghide uses the DCT coefficient technique for JPEG files and LSB (Least Significant Bit) substitution for BMP files — embedding data by making imperceptible changes to pixel values. With the correct passphrase the hidden file extracts cleanly.

Flag

The extracted file contains the picoCTF flag. The last two segments are obfuscated below — work through the steps above to recover them yourself.

```
picoCTF{h1dd3n_1n_pLa1n_XXXXX_XXXXXXXX}
```

> Hint: The penultimate segment is a leetspeak word hinting at visibility. The final segment is an 8-digit numeric challenge ID.

Key Takeaways

Concept	What We Used
Protocol Analysis	Identified TFTP as the file transfer mechanism from port 69 traffic
Packet Reassembly	Extracted raw file payloads from UDP data fields using tshark field extraction
Classical Cipher	Decoded ROT13-encoded instruction files to reveal tool and passphrase
Package Extraction	Used dpkg -x to extract a binary and its dependencies without root
Library Resolution	Set LD_LIBRARY_PATH to load custom shared libraries for steghide
Steganography	Used steghide to extract data hidden inside picture1.bmp

The core lesson: every layer of this challenge was individually weak. The files were transferred in cleartext over TFTP, the encoding was trivial ROT13, and the steghide passphrase was stored alongside the tool used to set it. Real-world data exfiltration often relies on the same assumption — that security through obscurity is sufficient. It isn't.